VISVESVARAYA TECHNOLOGICAL UNIVERSITY
"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

OPERATING SYSTEMS

Submitted by

Pratyush Ojha (1WA24CS216)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Feb-2026 to June-2026

Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by Pratyush Ojha (1WA24CS216), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026
. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil Assistant Professor Department of CSE BMSCE, Bengaluru	Dr. Kavitha Sooda Professor and Head Department of CSE BMSCE, Bengaluru

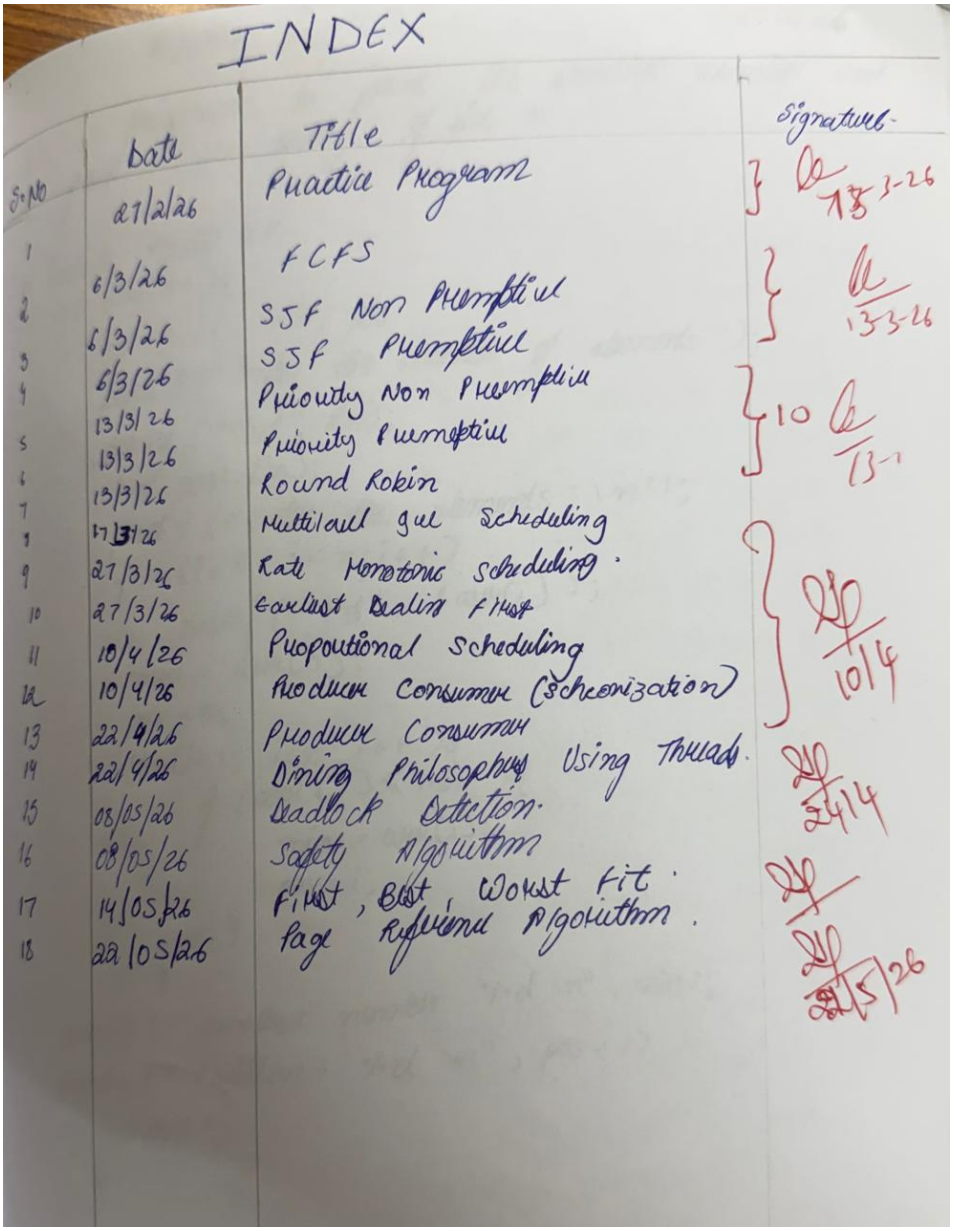
Index Sheet

Sl. No.	Experiment Title	Page No.

1.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF (Preemptive and Non-Preemptive) c) Priority (Preemptive and Non-Preemptive) d) Round Robin (Experiment with different quantum sizes for RR algorithm)	1-12
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	13-16
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	17-23
4.	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	24-28
5.	Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	29-34
6.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	35-38
7.	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	39-42

Course Outcomes:

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.



INDEX

S.No	Date	Title	Signature
1	27/2/26	Practice Program	13-3-26
2	6/3/26	FCFS	13-3-26
3	6/3/26	SJF Non Preemptive	
4	6/3/26	SJF Preemptive	
5	13/3/26	Priority Non Preemptive	13-3
6	13/3/26	Priority Preemptive	
7	13/3/26	Round Robin	
8	17/3/26	Multilevel queue Scheduling	
9	27/3/26	Rate Monotonic Scheduling.	
10	27/3/26	Earliest Deadline First	
11	10/4/26	Proportional Scheduling	10/4
12	10/4/26	Producer Consumer (Synchronization)	
13	22/4/26	Producer Consumer	
14	22/4/26	Dining Philosophers Using Threads.	24/4
15	08/05/26	Deadlock Detection.	
16	08/05/26	Safety Algorithm	
17	14/05/26	First, Best, Worst Fit.	
18	22/05/26	Page Replacement Algorithm.	28/5/26

Program 1

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one)
a) FCFS b) SJF c) Priority d) Round Robin (Experiment with different quantum sizes for RR algorithm)

FCFS:

```
#include <stdio.h>
int main() {
int n, i;
int at[20], bt[20], wt[20], tat[20], ct[20];
printf("1WA24CS216\n");
printf("Enter number of processes: ");
scanf("%d", &n);
printf("Enter Arrival Time and Burst Time:\n");
for(i = 0; i < n; i++) {
printf("P%d Arrival Time: ", i+1);
scanf("%d", &at[i]);
printf("P%d Burst Time: ", i+1);
scanf("%d", &bt[i]);
}
ct[0] = at[0] + bt[0];
for(i = 1; i < n; i++) {
if(ct[i-1] < at[i])
ct[i] = at[i] + bt[i];
else
ct[i] = ct[i-1] + bt[i];
}
for(i = 0; i < n; i++)
tat[i] = ct[i] - at[i];
1for(i = 0; i < n; i++)
wt[i] = tat[i] - bt[i];
float avg_wt = 0, avg_tat = 0;
for(i = 0; i < n; i++) {
avg_wt += wt[i];
avg_tat += tat[i];
}
printf("\nAverage Waiting Time = %.2f", avg_wt/n);
printf("\nAverage Turnaround Time = %.2f", avg_tat/n);
return 0;
}
```

```
pratyushojha@Pratyushs-MacBook-Pro Coding % cd "/Users/pratyushojha/Documents/Coding/" && gcc oslab.c -o oslab
&& "/Users/pratyushojha/Documents/Coding/"oslab
1WA24CS216
Enter number of processes: 4
Enter Arrival Time and Burst Time:
P1 Arrival Time: 1
P1 Burst Time: 3
P2 Arrival Time: 4
P2 Burst Time: 8
P3 Arrival Time: 2
P3 Burst Time: 3
P4 Arrival Time: 5
P4 Burst Time: 4

Average Waiting Time = 5.00
Average Turnaround Time = 9.50%
pratyushojha@Pratyushs-MacBook-Pro Coding %
```

SJF Non Preemptive:

```
#include <stdio.h>
int main() {
int n, i, j;
printf("1WA24CS216\n");
printf("Enter number of processes: ");
scanf("%d", &n);
int at[n], bt[n], ct[n], tat[n], wt[n];
int finished[n];
for(i = 0; i < n; i++) {
printf("Enter Arrival Time and Burst Time for P%d: ", i+1);
scanf("%d %d", &at[i], &bt[i]);
finished[i] = 0;
}
int current_time = 0;
int completed = 0;
while(completed < n) {
int min_bt = 9999;
int index = -1;
for(i = 0; i < n; i++) {
if(at[i] <= current_time && finished[i] == 0) {
if(bt[i] < min_bt) {
min_bt = bt[i];
index = i;
}
}
}
if(index == -1) {
current_time++;
}
else {
```

```
ct[index] = current_time + bt[index];
tat[index] = ct[index] - at[index];
wt[index] = tat[index] - bt[index];
current_time = ct[index];
finished[index] = 1;
completed++;
}
}
3printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
for(i = 0; i < n; i++) {
printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}
return 0;
}
```

```
pratyushojha@Pratyushs-MacBook-Pro Coding % cd "/Users/pratyushojha/Documents/Coding/" && gcc oslab.c -o oslab
&& "/Users/pratyushojha/Documents/Coding/"oslab
1WA24CS216
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 7
Enter Arrival Time and Burst Time for P2: 8 3
Enter Arrival Time and Burst Time for P3: 3 4
Enter Arrival Time and Burst Time for P4: 5 6

Process AT      BT      CT      TAT     WT
P1      0       7       7       7       0
P2      8       3       14      6       3
P3      3       4       11      8       4
P4      5       6       20      15      9
pratyushojha@Pratyushs-MacBook-Pro Coding %
```

SJF Preemptive:

```
#include <stdio.h>
int main() {
int n, i;
printf("1WA24CS16\n");
printf("Enter number of processes: ");
scanf("%d", &n);
int at[n], bt[n], rt[n], ct[n], tat[n], wt[n];
for(i = 0; i < n; i++) {
printf("Enter Arrival Time and Burst Time for P%d: ", i+1);
scanf("%d %d", &at[i], &bt[i]);
rt[i] = bt[i];
}
int current_time = 0, completed = 0;
while(completed < n) {
int min_rt = 9999, index = -1;
for(i = 0; i < n; i++) {
if(at[i] <= current_time && rt[i] > 0) {
if(rt[i] < min_rt) {
min_rt = rt[i];
index = i;
}
}
}
if(index == -1) {
current_time++;
}
else {
rt[index]--;
current_time++;
if(rt[index] == 0) {
completed++;
ct[index] = current_time;
tat[index] = ct[index] - at[index];
wt[index] = tat[index] - bt[index];
}
}
}
printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
5for(i = 0; i < n; i++) {
```

```
printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}
return 0;
}
```

```
pratyushojha@Pratyushs-MacBook-Pro Coding % cd "/Users/pratyushojha/Documents/Coding/" && gcc oslab.c -o oslab
&& "/Users/pratyushojha/Documents/Coding/"oslab
1WA24CS16
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 8
Enter Arrival Time and Burst Time for P2: 1 4
Enter Arrival Time and Burst Time for P3: 2 9
Enter Arrival Time and Burst Time for P4: 3 5

P       AT      BT      CT      TAT     WT
P1      0       8       17      17      9
P2      1       4       5       4       0
P3      2       9       26      24      15
P4      3       5       10      7       2
pratyushojha@Pratyushs-MacBook-Pro Coding %
```

Priority Non Preemptive:

```
#include <stdio.h>
int main() {
int n, i;
int pid[20], at[20], bt[20], pr[20];
int wt[20], tat[20], ct[20];
int completed[20] = {0};
int current_time = 0, completed_count = 0;
int highest_priority, selected;
float avg_wt = 0, avg_tat = 0;
printf("1WA24CS216\n");
printf("Enter number of processes: ");
scanf("%d", &n);
for(i = 0; i < n; i++) {
pid[i] = i + 1;
printf("P%d Arrival Time: ", i+1);
scanf("%d", &at[i]);
printf("P%d Burst Time: ", i+1);
scanf("%d", &bt[i]);
printf("P%d Priority: ", i+1);
scanf("%d", &pr[i]);
}
while(completed_count < n) {
highest_priority = 999;
selected = -1;
for(i = 0; i < n; i++) {
if(at[i] <= current_time && completed[i] == 0) {
if(pr[i] < highest_priority) {
highest_priority = pr[i];
selected = i;
}
if(selected == -1) {
current_time++;
}
}
}
else {
ct[selected] = current_time + bt[selected];
tat[selected] = ct[selected] - at[selected];
```

```
7wt[selected] = tat[selected] - bt[selected];
current_time = ct[selected];
completed[selected] = 1;
completed_count++;
avg_wt += wt[selected];
avg_tat += tat[selected];
}
}
printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n");
for(i = 0; i < n; i++) {
printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
pid[i], at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
}
avg_wt /= n;
avg_tat /= n;
printf("\nAverage Waiting Time = %.2f", avg_wt);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
return 0;
}
```

```
pratyushojha@Pratyushs-MacBook-Pro Coding % cd "/Users/pratyushojha/Documents/Coding/" && gcc oslab.c -o oslab
&& "/Users/pratyushojha/Documents/Coding/"oslab
P1 Arrival Time: 0
P1 Burst Time: 3
P1 Priority: 5
P2 Arrival Time: 2
P2 Burst Time: 2
P2 Priority: 3
P3 Arrival Time: 3
P3 Burst Time: 5
P3 Priority: 2
P4 Arrival Time: 3
P4 Burst Time: 4
P4 Priority: 4
P5 Arrival Time: 6
P5 Burst Time: 1
P5 Priority: 1

Process AT      BT      PR      CT      TAT     WT
P1      0       3       5       3       3       0
P2      2       2       3       11      9       7
P3      3       5       2       8       5       0
P4      3       4       4       15      12      8
P5      6       1       1       9       3       2

Average Waiting Time = 3.40
Average Turnaround Time = 6.40
pratyushojha@Pratyushs-MacBook-Pro Coding %
```

Priority Preemptive:

```
#include <stdio.h>
int main() {
int n, i, time = 0, completed = 0;
int pid[20], at[20], bt[20], pr[20];
int rt[20], ct[20], tat[20], wt[20];
int highest, pos;
float avg_wt = 0, avg_tat = 0;
printf("1WA24CS216\n");
printf("Enter number of processes: ");
scanf("%d", &n);
for(i = 0; i < n; i++) {
pid[i] = i + 1;
printf("P%d Arrival Time: ", i+1);
scanf("%d", &at[i]);
printf("P%d Burst Time: ", i+1);
scanf("%d", &bt[i]);
printf("P%d Priority: ", i+1);
scanf("%d", &pr[i]);
rt[i] = bt[i];
}
while(completed < n) {
highest = 999;
pos = -1;
for(i = 0; i < n; i++) {
if(at[i] <= time && rt[i] > 0 && pr[i] < highest) {
```

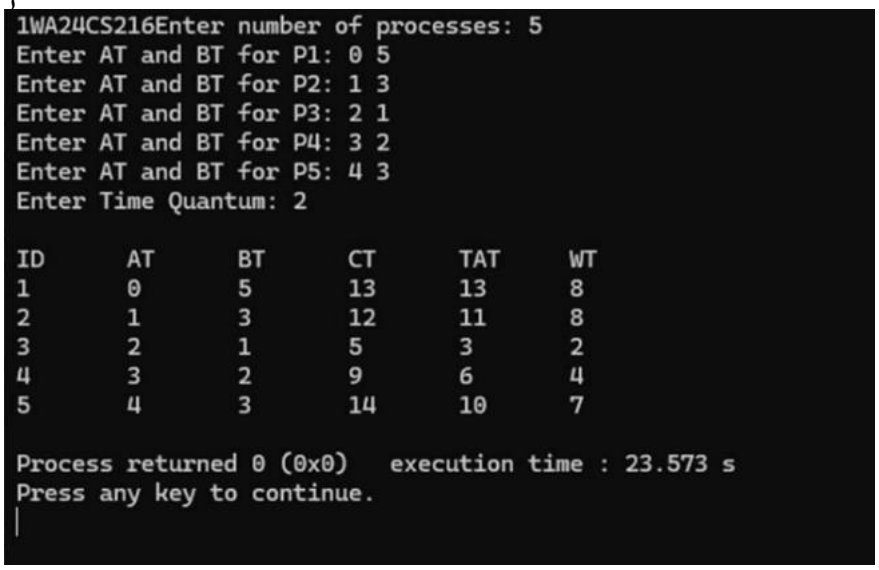
```
highest = pr[i];
pos = i;
}
}
if(pos == -1) {
time++;
}
else {
rt[pos]--;
time++;
if(rt[pos] == 0) {
completed++;
ct[pos] = time;
9tat[pos] = ct[pos] - at[pos];
wt[pos] = tat[pos] - bt[pos];
avg_wt += wt[pos];
avg_tat += tat[pos];
}
}
}
printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n");
for(i = 0; i < n; i++) {
printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
pid[i], at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
}
avg_wt /= n;
avg_tat /= n;
printf("\nAverage Waiting Time = %.2f", avg_wt);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
return 0;
}
```

```
1WA24CS216
Enter number of processes: 5
P1 Arrival Time: 0
P1 Burst Time: 3
P1 Priority: 3
P2 Arrival Time: 1
P2 Burst Time: 4
P2 Priority: 2
P3 Arrival Time: 2
P3 Burst Time: 6
P3 Priority: 4
P4 Arrival Time: 3
P4 Burst Time: 4
P4 Priority: 6
P5 Arrival Time: 4
P5 Burst Time: 2
P5 Priority: 10

Process AT      BT      PR      CT      TAT     WT
P1      0       3       3       7       7       4
P2      1       4       2       5       4       0
P3      2       6       4       13      11      5
P4      3       4       6       17      14      10
P5      4       2       10      19      15      13

Average Waiting Time = 6.40
Average Turnaround Time = 10.20

Process returned 0 (0x0)   execution time : 25.404 s
Press any key to continue.
```

Round Robin:

```
#include <stdio.h>
int main() {
int n, quantum, time = 0, completed = 0;
int queue[100], front = 0, rear = 0;
int visited[100] = {0};
printf("1WA24CS216\n");
printf("Enter number of processes: ");
scanf("%d", &n);
int at[n], bt[n], rt[n], ct[n], tat[n], wt[n];
for (int i = 0; i < n; i++) {
printf("Enter AT and BT for P%d: ", i + 1);
scanf("%d %d", &at[i], &bt[i]);
rt[i] = bt[i];
}
printf("Enter Time Quantum: ");
scanf("%d", &quantum);
for(int i = 0; i < n; i++) {
if(at[i] <= time) {
queue[rear++] = i;
visited[i] = 1;
}
}
while (front < rear) {
int i = queue[front++];
int execute = (rt[i] > quantum) ? quantum : rt[i];
rt[i] -= execute;
time += execute;
for (int j = 0; j < n; j++) {
if (at[j] <= time && visited[j] == 0) {
queue[rear++] = j;
visited[j] = 1;
}
}
if (rt[i] > 0) {
queue[rear++] = i;
}
else {
ct[i] = time;
11completed++;
}
if (front == rear && completed < n) {
for (int j = 0; j < n; j++) {
if (visited[j] == 0) {
time = at[j];
```

```
queue[rear++] = j;
visited[j] = 1;
break;
}
}
}
}
printf("\nID\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
printf("%d\t%d\t%d\t%d\t%d\t%d\n",
i+1, at[i], bt[i], ct[i],
ct[i]-at[i],
(ct[i]-at[i])-bt[i]);
}
return 0;
}
```



Program 2

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```
#include <stdio.h>
#include <string.h>
#define MAX 100
struct Process {
char pid[5];
int arrival, burst, remaining;
int completion, turnaround, waiting;
int qno;
int done;
};
int main() {
struct Process p[MAX];
int n, i, completed = 0;
int current_time = 0, tq;
float total_tat = 0, total_wt = 0;
printf("1WA24CS216\n");
printf("Enter number of processes: ");
scanf("%d", &n);
for(i = 0; i < n; i++) {
printf(p[i].pid, "p%d", i+1);
printf("\nProcess %s\n", p[i].pid);
printf("Enter Arrival Time: ");
scanf("%d", &p[i].arrival);
printf("Enter Burst Time: ");
```

```
scanf("%d", &p[i].burst);
p[i].remaining = p[i].burst;
p[i].done = 0;
printf("Enter Queue (1-System, 2-User): ");
scanf("%d", &p[i].qno);
}
printf("\nEnter Time Quantum for System Queue: ");
scanf("%d", &tq);
while(completed < n) {
int executed = 0;
13for(i = 0; i < n; i++) {
if(p[i].qno == 1 && p[i].arrival <= current_time && p[i].remaining > 0) {
executed = 1;
if(p[i].remaining > tq) {
p[i].remaining -= tq;
current_time += tq;
}
else {
current_time += p[i].remaining;
p[i].remaining = 0;
p[i].completion = current_time;
p[i].turnaround = p[i].completion - p[i].arrival;
p[i].waiting = p[i].turnaround - p[i].burst;
total_tat += p[i].turnaround;
total_wt += p[i].waiting;
completed++;
}
}
}
if(executed)
continue;
int idx = -1;
int min_arrival = 1e9;
for(i = 0; i < n; i++) {
if(p[i].qno == 2 && p[i].arrival <= current_time && p[i].remaining > 0) {
if(p[i].arrival < min_arrival) {
min_arrival = p[i].arrival;
idx = i;
}
}
}
if(idx != -1) {
executed = 1;
int run_time = p[idx].remaining;
for(int t = 1; t <= run_time; t++) {
```

```
current_time++;
p[idx].remaining--;
int sys_arrived = 0;
for(i = 0; i < n; i++) {
if(p[i].qno == 1 && p[i].arrival <= current_time && p[i].remaining > 0) {
14sys_arrived = 1;
break;
}
}
if(sys_arrived)
break;
}
if(p[idx].remaining == 0) {
p[idx].completion = current_time;
p[idx].turnaround = p[idx].completion - p[idx].arrival;
p[idx].waiting = p[idx].turnaround - p[idx].burst;
total_tat += p[idx].turnaround;
total_wt += p[idx].waiting;
completed++;
}
}
if(!executed)
current_time++;
}
printf("\nPID\tQ\tAT\tBT\tCT\tTAT\tWT\n");
for(i = 0; i < n; i++) {
printf("%s\t%d\t%d\t%d\t%d\t%d\t%d\n",
p[i].pid, p[i].qno, p[i].arrival, p[i].burst,
p[i].completion, p[i].turnaround, p[i].waiting);
}
printf("\nAverage Turnaround Time = %.2f", total_tat / n);
printf("\nAverage Waiting Time = %.2f\n", total_wt / n);
return 0;
}
```

```
1WA24CS216
Enter number of processes: 4

Process
Enter Arrival Time: 0
Enter Burst Time: 4
Enter Queue (1-System, 2-User): 1
    R
Process     R
Enter Arrival Time: 0
Enter Burst Time: 3
Enter Queue (1-System, 2-User): 1

Process
Enter Arrival Time: 0
Enter Burst Time: 8
Enter Queue (1-System, 2-User): 2

Process
Enter Arrival Time: 10
Enter Burst Time: 5
Enter Queue (1-System, 2-User): 1

Enter Time Quantum for System Queue: 2

PID     Q       AT      BT      CT      TAT     WT
        1       0       4       6       6       2
    R   1       0       3       7       7       4
        2       0       8       20      20      12
        1       10      5       15      5       0

Average Turnaround Time = 9.50
Average Waiting Time = 4.50

Process returned 0 (0x0)   execution time : 35.080 s
Press any key to continue.
```

Program 3

Write a C program to simulate Real-Time CPU Scheduling algorithms: (Any one)
a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling

Rate Monotonic:

```
#include <stdio.h>
int gcd(int a, int b) {
while (b) {
int t = b;
b = a % b;
a = t;
}
return a;
}
int lcm(int a, int b) {
return (a * b) / gcd(a, b);
}
struct Task {
int id;
int burst;
int period;
int remaining;
};
int main() {
int n;
printf("1WA24CS216\n");
printf("Enter number of processes: ");
scanf("%d", &n);
struct Task t[n];
printf("Enter burst times:\n");
for (int i = 0; i < n; i++) {
scanf("%d", &t[i].burst);
t[i].id = i + 1;
}
printf("Enter periods:\n");
for (int i = 0; i < n; i++) {
scanf("%d", &t[i].period);
t[i].remaining = 0;
}
17for (int i = 0; i < n - 1; i++) {
for (int j = i + 1; j < n; j++) {
if (t[i].period > t[j].period) {
struct Task temp = t[i];
t[i] = t[j];
t[j] = temp;
```

```
}
}
}
int hyper = t[0].period;
for (int i = 1; i < n; i++)
hyper = lcm(hyper, t[i].period);
float util = 0;
for (int i = 0; i < n; i++)
util += (float)t[i].burst / t[i].period;
float bound;
if (n == 1)
bound = 1.0;
else if (n == 2)
bound = 0.828427;
else if (n == 3)
bound = 0.779763;
else
bound = 0.693;
printf("\n%f <= %f => %s\n", util, bound, (util <= bound) ? "true" : "false");
printf("\nScheduling occurs for %d ms\n\n", hyper);
int remaining[n];
for (int i = 0; i < n; i++)
remaining[i] = 0;
int last = -2;
for (int time = 0; time < hyper; time++) {
for (int i = 0; i < n; i++) {
if (time % t[i].period == 0)
remaining[i] = t[i].burst;
}
int run = -1;
for (int i = 0; i < n; i++) {
if (remaining[i] > 0) {
run = i;
break;
18}
}
if (run != last) {
if (run != -1)
else
printf("%dms onwards: Process %d running\n", time, t[run].id);
printf("%dms onwards: CPU is idle\n", time);
}
if (run != -1)
remaining[run]--;
last = run;
```

```
}
return 0;
}
```

```
pratyushojha@Pratyushs-MacBook-Pro Coding % cd "/Users/pratyushojha/Documents/Coding/" && gcc oslab.c -o oslab
&& "/Users/pratyushojha/Documents/Coding/"oslab
1WA24CS216
Enter number of processes: 3
Enter burst times:
3 2 2
Enter periods:
20 5 10

0.750000 <= 0.779763 => true

Scheduling occurs for 20 ms

0ms onwards: Process 2 running
2ms onwards: Process 3 running
4ms onwards: Process 1 running
5ms onwards: Process 2 running
7ms onwards: Process 1 running
9ms onwards: CPU is idle
10ms onwards: Process 2 running
12ms onwards: Process 3 running
14ms onwards: CPU is idle
15ms onwards: Process 2 running
17ms onwards: CPU is idle
pratyushojha@Pratyushs-MacBook-Pro Coding %
```

Earliest Deadline First:

```
#include <stdio.h>
#include <limits.h>
typedef struct {
int id;
int burst;
int deadline;
int period;
int remaining_burst;
int next_deadline;
} Task;
int main() {
int n, total_time;
printf("1WA24CS216\n");
printf("Enter number of tasks: ");
scanf("%d", &n);
Task tasks[n];
for (int i = 0; i < n; i++) {
printf("Enter PID, Burst, Deadline, and Period for Task %d: ", i + 1);
scanf("%d %d %d %d",
&tasks[i].id,
&tasks[i].burst,
```

```
&tasks[i].deadline,
&tasks[i].period);
tasks[i].remaining_burst = 0;
tasks[i].next_deadline = 0;
}
printf("Enter total scheduling time (ms): ");
scanf("%d", &total_time);
printf("\nScheduling occurs for %d ms\n\n", total_time);
for (int t = 0; t < total_time; t++) {
for (int i = 0; i < n; i++) {
if (t % tasks[i].period == 0) {
tasks[i].remaining_burst = tasks[i].burst;
tasks[i].next_deadline = t + tasks[i].deadline;
}
}
int selected = -1;
int min_deadline = INT_MAX;
20for (int i = 0; i < n; i++) {
if (tasks[i].remaining_burst > 0) {
if (tasks[i].next_deadline < min_deadline) {
min_deadline = tasks[i].next_deadline;
selected = i;
}
else if (tasks[i].next_deadline == min_deadline) {
if (selected == -1 || tasks[i].id < tasks[selected].id) {
selected = i;
}
}
if (selected == -1) {
printf("%dms : CPU is idle.\n", t);
}
}
}
else {
printf("%dms : Task %d is running.\n", t, tasks[selected].id);
tasks[selected].remaining_burst--;
}
}
return 0;
}
```

```
1WA24CS216
Enter number of tasks: 3
Enter PID, Burst, Deadline, and Period for Task 1: 1 3 7 20
Enter PID, Burst, Deadline, and Period for Task 2: 2 2 4 5
Enter PID, Burst, Deadline, and Period for Task 3: 3 2 8 10
Enter total scheduling time (ms): 20

Scheduling occurs for 20 ms

0ms : Task 2 is running.
1ms : Task 2 is running.
2ms : Task 1 is running.
3ms : Task 1 is running.
4ms : Task 1 is running.
5ms : Task 3 is running.
6ms : Task 3 is running.
7ms : Task 2 is running.
8ms : Task 2 is running.
9ms : CPU is idle.
10ms : Task 2 is running.
11ms : Task 2 is running.
12ms : Task 3 is running.
13ms : Task 3 is running.
14ms : CPU is idle.
15ms : Task 2 is running.
16ms : Task 2 is running.
17ms : CPU is idle.
18ms : CPU is idle.
19ms : CPU is idle.

Process returned 0 (0x0)   execution time : 23.386 s
Press any key to continue.
```

Proportional Scheduling:

```
#include<stdio.h>
#include <stdlib.h>
#include <time.h>
```

```
typedef struct {
char name[5];
int tickets;
} Process;
int main() {
int n, total_tickets = 0;
float total_T = 0.0;
printf("1WA24CS216\n");
printf("Enter the number of Processes: ");
scanf("%d", &n);
Process p[n];
srand(time(NULL));
for (int i = 0; i < n; i++) {
printf("\nProcess %d:\n", i + 1);
sprintf(p[i].name, "P%d", i + 1);
printf("Tickets: ");
scanf("%d", &p[i].tickets);
total_tickets += p[i].tickets;
total_T += p[i].tickets;
}
printf("\n--- Proportional Share Scheduling ---\n");
printf("Enter the Time Period for scheduling: ");
int m;
scanf("%d", &m);
for (int i = 0; i < m; i++) {
int winning_ticket = rand() % total_tickets + 1;
int accumulated_tickets = 0;
int winner_index;
for (int j = 0; j < n; j++) {
accumulated_tickets += p[j].tickets;
if (winning_ticket <= accumulated_tickets) {
winner_index = j;
break;
}
}
22printf("Tickets picked: %d, Winner: %s\n",
winning_ticket,
p[winner_index].name);
}
for (int i = 0; i < n; i++) {
printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n",
p[i].name,
((p[i].tickets / total_T) * 100));
}
return 0;
```

}

```
1WA24CS216
Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 59, Winner: P3
Tickets picked: 60, Winner: P3
Tickets picked: 34, Winner: P3
Tickets picked: 26, Winner: P2
Tickets picked: 31, Winner: P3

The Process: P1 gets 16.67% of Processor Time.

The Process: P2 gets 33.33% of Processor Time.

The Process: P3 gets 50.00% of Processor Time.

Process returned 0 (0x0)   execution time : 4.931 s
Press any key to continue.
```

Program 4

Write a C program to simulate: (Any one)

a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem

Producer Consumer:

```
#include <stdio.h>
#include <stdlib.h>
#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int in = 0, out = 0;
int mutex = 1;
int empty = BUFFER_SIZE;
int full = 0;
int item = 0;
void wait(int *s) {
(*s)--;
}
void signal(int *s) {
(*s)++;
}
void produce() {
if (empty == 0) {
printf("Buffer is full\n");
return;
}
wait(&empty);
wait(&mutex);
item++;
buffer[in] = item;
in = (in + 1) % BUFFER_SIZE;
printf("Producer has produced: item%d\n", item);
signal(&mutex);
signal(&full);
}
void consume() {
if (full == 0) {
printf("Buffer is empty\n");
return;
}
wait(&full);
wait(&mutex);
int consumed = buffer[out];
out = (out + 1) % BUFFER_SIZE;
printf("Consumer has consumed: item%d\n", consumed);
```

```
signal(&mutex);
signal(&empty);
}
int main() {
int choice;
printf("\n1WA24CS216");
while (1) {
printf("\nEnter your choice:\n");
printf("1. Producer\n2. Consumer\n3. Exit\n");
scanf("%d", &choice);
switch (choice) {
case 1:
produce();
break;
case 2:
consume();
break;
case 3:
exit(0);
default:
printf("Invalid choice\n");
}
}
return 0;
}
```

```
pratyushojha@Pratyushs-MacBook-Pro Coding % cd "/Users/pratyushojha/Documents/Coding/" && gcc oslab.c
&& "/Users/pratyushojha/Documents/Coding/"oslab
1WA24CS216

1. Producer
2. Consumer
3. Exit
Enter choice: 1
Producer has produced: item1
1WA24CS216

1. Producer
2. Consumer
3. Exit
Enter choice: 1
Producer has produced: item2
1WA24CS216

1. Producer
2. Consumer
3. Exit
Enter choice: 2
Consumer has consumed: item1
1WA24CS216

1. Producer
2. Consumer
3. Exit
Enter choice: 2
Consumer has consumed: item2
1WA24CS216

1. Producer
2. Consumer
3. Exit
Enter choice: 2
Buffer is empty
1WA24CS216

1. Producer
2. Consumer
3. Exit
Enter choice: 3
pratyushojha@Pratyushs-MacBook-Pro Coding %
```

Dining Philosopher Problem:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#define N 5
pthread_mutex_t forks[N];
pthread_t philosophers[N];
void* philosopher(void* num)
```

```
{
int id = *(int*)num;
int left = id;
int right = (id + 1) % N;
while (1)
{
printf("Philosopher %d is thinking.\n", id);
sleep(1);
if (id % 2 == 0)
{
pthread_mutex_lock(&forks[left]);
printf("Philosopher %d picked left fork %d\n", id, left);
pthread_mutex_lock(&forks[right]);
printf("Philosopher %d picked right fork %d\n", id, right);
}
else
{
pthread_mutex_lock(&forks[right]);
printf("Philosopher %d picked right fork %d\n", id, right);
pthread_mutex_lock(&forks[left]);
printf("Philosopher %d picked left fork %d\n", id, left);
}
printf("Philosopher %d is eating.\n", id);
sleep(2);
pthread_mutex_unlock(&forks[left]);
pthread_mutex_unlock(&forks[right]);
printf("Philosopher %d put down forks %d and %d\n", id, left, right);
}
return NULL;
}
27int main()
{
int i;
int ids[N];
for (i = 0; i < N; i++)
{
pthread_mutex_init(&forks[i], NULL);
ids[i] = i;
}
for (i = 0; i < N; i++)
{
pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
}
for (i = 0; i < N; i++)
{
```

```
pthread_join(philosophers[i], NULL);
}
for (i = 0; i < N; i++)
{
pthread_mutex_destroy(&forks[i]);
}
return 0;
}
```

```
1WA24CS216
Philosopher 0 is thinking.
Philosopher 3 is thinking.
Philosopher 2 is thinking.
Philosopher 1 is thinking.
Philosopher 4 is thinking.
Philosopher 4 picked up left fork 4.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 0 picked up left fork 0.
Philosopher 1 put down forks 1 and 2.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 1 is thinking.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 1 picked up right fork 2.
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
```

Program 5

Write a C program to simulate: (Any one)

a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection

Banker's Algorithm:

```
#include <stdio.h>
int main() {
int n, m, i, j, k;
printf("1WA24CS216\n");
printf("Enter number of processes: ");
scanf("%d", &n);
printf("Enter number of resources: ");
scanf("%d", &m);
int allocation[n][m];
int max[n][m];
int need[n][m];
int available[m];
int work[m];
int finish[n];
int safeSequence[n];
printf("\nEnter Allocation Matrix:\n");
for(i = 0; i < n; i++) {
for(j = 0; j < m; j++) {
scanf("%d", &allocation[i][j]);
}
}
printf("\nEnter Maximum Demand Matrix:\n");
for(i = 0; i < n; i++) {
for(j = 0; j < m; j++) {
scanf("%d", &max[i][j]);
}
}
printf("\nEnter Available Resources:\n");
for(i = 0; i < m; i++) {
scanf("%d", &available[i]);
work[i] = available[i];
}
for(i = 0; i < n; i++) {
for(j = 0; j < m; j++) {
29need[i][j] = max[i][j] - allocation[i][j];
```

```
}
}
for(i = 0; i < n; i++) {
finish[i] = 0;
}
int count = 0;
while(count < n) {
int found = 0;
for(i = 0; i < n; i++) {
if(finish[i] == 0) {
int possible = 1;
for(j = 0; j < m; j++) {
if(need[i][j] > work[j]) {
possible = 0;
break;
}
}
if(possible) {
for(k = 0; k < m; k++) {
work[k] += allocation[i][k];
}
safeSequence[count] = i;
count++;
finish[i] = 1;
found = 1;
}
}
}
if(found == 0)
break;
}
int safe = 1;
for(i = 0; i < n; i++) {
if(finish[i] == 0) {
safe = 0;
break;
}
}
if(safe) {
30printf("\nSystem is in a safe state.\n");
printf("Safe sequence is: ");
for(i = 0; i < n; i++) {
printf("P%d", safeSequence[i]);
if(i != n - 1)
printf(" -> ");
```

```
printf("\n");
}
}
else {
}
return 0;
printf("\nSystem is not in a safe state.\n");
}
```

```
1WA24CS216
Enter number of processes: 5
Enter number of resources: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Maximum Demand Matrix:
7 5 3
3 2 2

9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence is: P1 ->
P3 ->
P4 ->
P0 ->
P2
pratyushojha@Pratyushs-MacBook-Pro Coding %
```

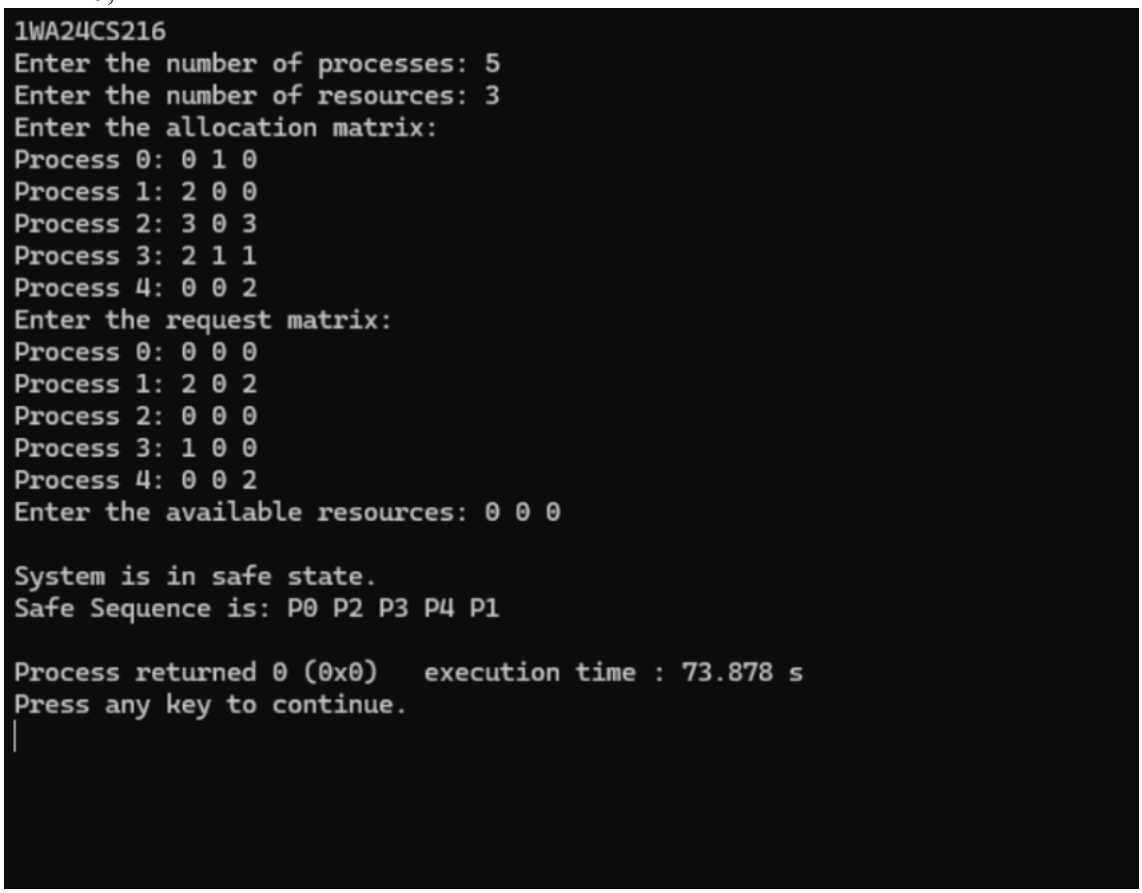
Deadlock Detection:

```
#include <stdio.h>
int main() {
int n, m, i, j, k;
printf("1WA24CS216\n");
printf("Enter the number of processes: ");
scanf("%d", &n);
printf("Enter the number of resources: ");
```

```
scanf("%d", &m);
int allocation[n][m];
int request[n][m];
int available[m];
int work[m];
int finish[n];
int safeSequence[n];
printf("Enter the allocation matrix:\n");
for(i = 0; i < n; i++) {
printf("Process %d: ", i);
for(j = 0; j < m; j++) {
scanf("%d", &allocation[i][j]);
}
}
printf("Enter the request matrix:\n");
for(i = 0; i < n; i++) {
printf("Process %d: ", i);
for(j = 0; j < m; j++) {
scanf("%d", &request[i][j]);
}
}
printf("Enter the available resources: ");
for(i = 0; i < m; i++) {
scanf("%d", &available[i]);
work[i] = available[i];
}
for(i = 0; i < n; i++) {
int allZero = 1;
for(j = 0; j < m; j++) {
if(allocation[i][j] != 0) {
allZero = 0;
32break;
}
}
if(allZero)
finish[i] = 1;
else
finish[i] = 0;
}
int count = 0;
while(count < n) {
int found = 0;
for(i = 0; i < n; i++) {
if(finish[i] == 0) {
int possible = 1;
```

```
for(j = 0; j < m; j++) {
if(request[i][j] > work[j]) {
possible = 0;
break;
}
}
if(possible) {
for(k = 0; k < m; k++) {
work[k] += allocation[i][k];
}
safeSequence[count] = i;
count++;
finish[i] = 1;
found = 1;
}
}
}
if(found == 0)
break;
}
int deadlock = 0;
for(i = 0; i < n; i++) {
if(finish[i] == 0) {
deadlock = 1;
break;
}
33}
}
if(deadlock) {
printf("\nSystem is in DEADLOCK state.\n");
printf("Deadlocked processes are: ");
for(i = 0; i < n; i++) {
if(finish[i] == 0)
printf("P%d ", i);
}
printf("\n");
}
else {
printf("\nSystem is in safe state.\n");
printf("Safe Sequence is: ");
for(i = 0; i < n; i++) {
printf("P%d ", safeSequence[i]);
}
printf("\n");
}
```

return 0;



Program 6

Write a C program to simulate the following contiguous memory allocation techniques. (Any one)

a) Worst-fit b) Best-fit c) First-fit

```
#include <stdio.h>
void firstFit(int blockSize[], int m, int processSize[], int n)
{
int allocation[50];
int used[50] = {0};
for(int i = 0; i < n; i++)
allocation[i] = -1;
for(int i = 0; i < n; i++)
{
```

```
for(int j = 0; j < m; j++)
{
if(blockSize[j] >= processSize[i] && used[j] == 0)
{
allocation[i] = j;
used[j] = 1;
break;
}
}
}
printf("\n--- First Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");
for(int i = 0; i < n; i++)
{
printf("%d\t\t%d\t\t", i + 1, processSize[i]);
if(allocation[i] != -1)
printf("%d\n", allocation[i] + 1);
else
printf("Not Allocated\n");
}
}
void bestFit(int blockSize[], int m, int processSize[], int n)
{
int allocation[50];
int used[50] = {0};
for(int i = 0; i < n; i++)
35allocation[i] = -1;
for(int i = 0; i < n; i++)
{
int bestIdx = -1;
for(int j = 0; j < m; j++)
{
if(blockSize[j] >= processSize[i] && used[j] == 0)
{
if(bestIdx == -1 || blockSize[j] < blockSize[bestIdx])
bestIdx = j;
}
}
if(bestIdx != -1)
{
allocation[i] = bestIdx;
used[bestIdx] = 1;
}
}
printf("\n--- Best Fit ---\n");
```

```
printf("Process No.\tProcess Size\tBlock No.\n");
for(int i = 0; i < n; i++)
{
printf("%d\t\t%d\t\t", i + 1, processSize[i]);
if(allocation[i] != -1)
printf("%d\n", allocation[i] + 1);
else
printf("Not Allocated\n");
}
}
void worstFit(int blockSize[], int m, int processSize[], int n)
{
int allocation[50];
int used[50] = {0};
for(int i = 0; i < n; i++)
allocation[i] = -1;
for(int i = 0; i < n; i++)
{
int worstIdx = -1;
for(int j = 0; j < m; j++)
{
if(blockSize[j] >= processSize[i] && used[j] == 0)
{
if(worstIdx == -1 || blockSize[j] > blockSize[worstIdx])
worstIdx = j;
}
}
if(worstIdx != -1)
{
allocation[i] = worstIdx;
used[worstIdx] = 1;
}
}
printf("\n--- Worst Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");
for(int i = 0; i < n; i++)
{
printf("%d\t\t%d\t\t", i + 1, processSize[i]);
if(allocation[i] != -1)
printf("%d\n", allocation[i] + 1);
else
printf("Not Allocated\n");
}
}
int main()
```

```
{
int m, n;
int blockSize[50], processSize[50];
printf("1WA24CS216\n");
printf("Enter number of memory blocks: ");
scanf("%d", &m);
printf("Enter sizes of %d memory blocks:\n", m);
for(int i = 0; i < m; i++)
scanf("%d", &blockSize[i]);
printf("Enter number of processes: ");
scanf("%d", &n);
printf("Enter sizes of %d processes:\n", n);
for(int i = 0; i < n; i++)
scanf("%d", &processSize[i]);
firstFit(blockSize, m, processSize, n);
bestFit(blockSize, m, processSize, n);
worstFit(blockSize, m, processSize, n);
return 0;
}
```

```
pratyushojha@Pratyushs-MacBook-Pro Coding % cd "/Users/pratyushojha/Documents/Coding/" && gcc oslab.c -o oslab
&& "/Users/pratyushojha/Documents/Coding/"oslab
1WA24CS216
Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes:
212 417 112 426

--- First Fit ---
Process No.     Process Size    Block No.
1               212             2
2               417             5
3               112             3
4               426             Not Allocated

--- Best Fit ---
Process No.     Process Size    Block No.
1               212             4
2               417             2
3               112             3
4               426             5

--- Worst Fit ---
Process No.     Process Size    Block No.
1               212             5
2               417             2
3               112             4
4               426             Not Allocated
pratyushojha@Pratyushs-MacBook-Pro Coding %
```

Program 7

Write a C program to simulate page replacement algorithms. (Any one)

a) FIFO b) LRU c) Optimal

```
#include <stdio.h>
#include <stdlib.h>
void simulate(int pages[], int n, int nf, int mode) {
int frames[10] = {0}, time[10] = {0}, faults = 0, next = 0;
for (int i = 0; i < nf; i++)
frames[i] = -1;
printf("\n--- %s Page Replacement ---\n",
mode == 0 ? "FIFO" : mode == 1 ? "LRU" : "Optimal");
for (int i = 0; i < n; i++) {
int hit = -1;
for (int j = 0; j < nf; j++)
if (frames[j] == pages[i])
hit = j;
if (hit != -1) {
if (mode == 1)
time[hit] = i;
}
else {
int rep = -1;
for (int j = 0; j < nf; j++)
if (frames[j] == -1) {
rep = j;
break;
}
if (rep == -1) {
if (mode == 0) {
```

```
rep = next;
next = (next + 1) % nf;
}
else {
int extreme = (mode == 1) ? i : -1;
for (int j = 0; j < nf; j++) {
int look = -1;
if (mode == 1)
look = time[j];
else {
39for (int k = i + 1; k < n; k++)
if (pages[k] == frames[j]) {
look = k;
break;
}
if (look == -1) {
rep = j;
break;
}
}
if ((mode == 1 && look < extreme) ||
(mode == 2 && look > extreme)) {
extreme = look;
rep = j;
}
}
}
}
frames[rep] = pages[i];
if (mode == 1)
time[rep] = i;
faults++;
}
printf("Page %d -> [", pages[i]);
for (int j = 0; j < nf && frames[j] != -1; j++)
printf("%d%s", frames[j],
(j < nf - 1 && frames[j + 1] != -1) ? " " : "");
printf("]\n");
}
printf("Total Page Faults (%s): %d\n",
mode == 0 ? "FIFO" : mode == 1 ? "LRU" : "Optimal",
faults);
}
int main() {
int n, nf, *pages;
```

```
printf("1WA24CS216\n");
printf("Enter number of pages: ");
scanf("%d", &n);
pages = (int*)malloc(n  * sizeof(int));
printf("Enter page reference string:\n");
40for (int i = 0; i < n; i++)
scanf("%d", &pages[i]);
printf("Enter number of frames: ");
scanf("%d", &nf);
for (int m = 0; m < 3; m++)
simulate(pages,  n, nf, m);
free(pages);
return 0;
}
```

```
pratyushojha@Pratyushs-MacBook-Pro Coding % cd "/Users/pratyushojha/Documents/Coding/" && gcc oslab.c -o oslab
&& "/Users/pratyushojha/Documents/Coding/"oslab
1WA24CS216
Enter number of pages: 15
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3 1 2 0
Enter number of frames: 3

--- FIFO Page Replacement ---
Page 7 -> [7]
Page 0 -> [7 0]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 3 1]
Page 0 -> [2 3 0]
Page 4 -> [4 3 0]
Page 2 -> [4 2 0]
Page 3 -> [4 2 3]
Page 0 -> [0 2 3]
Page 3 -> [0 2 3]
Page 1 -> [0 1 3]
Page 2 -> [0 1 2]
Page 0 -> [0 1 2]
Total Page Faults (FIFO): 12

--- LRU Page Replacement ---
Page 7 -> [7]
Page 0 -> [7 0]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 0 3]
Page 0 -> [2 0 3]
Page 4 -> [4 0 3]
Page 2 -> [4 0 2]
Page 3 -> [4 3 2]
Page 0 -> [0 3 2]
Page 3 -> [0 3 2]
Page 1 -> [0 3 1]
Page 2 -> [2 3 1]
Page 0 -> [2 0 1]
Total Page Faults (LRU): 12
```

```
--- Optimal Page Replacement ---
Page 7 -> [7]
Page 0 -> [7 0]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 0 3]
Page 0 -> [2 0 3]
Page 4 -> [2 4 3]
Page 2 -> [2 4 3]
Page 3 -> [2 4 3]
Page 0 -> [2 0 3]
Page 3 -> [2 0 3]
Page 1 -> [2 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Total Page Faults (Optimal): 8
pratyushojha@Pratyushs-MacBook-Pro Coding %
```